The Environment Semantics Gap in Structured CTI: Measuring SUT Requirements for APT Emulation

Sidnei Barbieri*†
Carnegie Mellon University
Pittsburgh, PA, USA
sbarbier@andrew.cmu.edu

Ágney Lopes Roth Ferraz*
Aeronautics Institute of Technology
São José dos Campos, SP, Brazil
roth@ita.br

Lourenço Alves Pereira Júnior
Aeronautics Institute of Technology
São José dos Campos, SP, Brazil
ljr@ita.br

Abstract

APT emulation fails when the target environment is wrong. Public CTI often describes adversary behavior, but it still leaves the *System Under Test* (SUT) underspecified. The missing detail spans operating system, software, vulnerability, and deployment context, so analysts and range builders still have to finish the environment by hand. We measure how much of that environment signal current structured CTI actually exposes in MITRE ATT&CK STIX bundles.

Across the ATT&CK Enterprise, Mobile, and ICS datasets, with CAPEC and FiGHT as coverage contrasts, we quantify platform coverage, software specificity, vulnerability evidence, and deployment compatibility. Platform tags are common, but software references rarely pin versions or Common Platform Enumeration (CPE) identifiers (97.6% do not), and vulnerability evidence remains sparse and fragmented.

This turns a familiar operational complaint into a measured corpus limit rather than a tooling complaint. Current structured CTI can narrow the environment and guide backend selection, but it cannot derive replay-ready SUTs from structured fields alone. Profile ambiguity also drops quickly once evidence density rises: in this corpus, confusion falls from 1.3% when profiles contain at least one linked software item to 0% once they contain at least two. These results establish a reproducible baseline for what current structured CTI can automate in SUT construction, what still returns to the analyst, and which schema changes would reduce that remainder.

CCS Concepts

• **Security and privacy → Intrusion/anomaly detection and malware mitigation**; *Systems security*; Network security.

Keywords

cyber threat intelligence, STIX, MITRE ATT&CK, system under test, adversary emulation, structured CTI

ACM Reference Format:
Sidnei Barbieri, Ágney Lopes Roth Ferraz, and Lourenço Alves Pereira Júnior. 2026. The Environment Semantics Gap in Structured CTI: Measuring SUT Requirements for APT Emulation. In *Proceedings of 2026 ACM SIGSAC Conference on Computer and Communications Security (ACM CCS '26).* ACM, New York, NY, USA, 15 pages.

*These authors contributed equally to this work.
†Also with Aeronautics Institute of Technology.

ACM CCS '26, The Hague, The Netherlands
2026.

1 Introduction

Controlled APT emulation needs two inputs: a description of *what* an adversary does and an environment in which those actions can run. The first is increasingly well served by the MITRE ATT&CK knowledge base, which now catalogues 835 Enterprise techniques across 187 documented groups and 52 campaigns, all encoded in machine-readable STIX bundles [23, 25]. The second remains underspecified. Each emulation run still needs a *System Under Test (SUT)*: a target environment with the right operating system, software stack, exposure surface, and vulnerability state. If that environment is wrong, even an accurate procedure produces either a silent no-op or a result that cannot be reproduced.

Rather than asking whether CTI can drive end-to-end emulation, this paper asks a narrower question: *how much of the target environment can current structured CTI determine on its own?* The input to that question is public structured CTI in ATT&CK-style STIX bundles. The output is a bounded SUT claim: what the corpus can already narrow, what backend family it can justify, and what detail still returns to the analyst. Success is not push-button SUT reconstruction, but a reproducible boundary between corpus-supported narrowing and analyst-supplied instantiation.

Across the ATT&CK Enterprise, Mobile, and ICS corpora, with CAPEC and FiGHT used as coverage contrasts, the pattern is consistent. Platform evidence is common but coarse. Software references rarely pin versions or Common Platform Enumeration (CPE) identifiers, and CVE evidence is sparse and often split between structured fields and free text. Public CTI can therefore narrow candidate environments and guide backend planning, but it does not derive replay-ready SUTs from structured fields alone.

That measured boundary is the paper's contribution. Practitioners already know that CTI-driven SUT construction is difficult, but the field still lacks a reproducible quantitative boundary between what current corpora encode and what analysts must still supply. That boundary matters to the people who actually build and evaluate emulation environments: analysts, range designers, emulation engineers, and CTI curators. Red-team platforms execute only after the environment has been declared [1, 4, 16, 17, 26], and weak software or vulnerability context degrades both detection evaluation and attribution claims [7, 9, 13, 14, 18, 28, 29]. This paper turns that workflow boundary into a measurable object.

ATT&CK is the right place to measure this gap: it is the most widely reused structured CTI source for threat-informed defense, yet its schema was designed to encode behavior rather than deployment-grade environment constraints [6, 20, 23, 25]. This paper therefore measures the environment gap directly rather than the procedural translation problem.

RQ1: How much structured CTI encodes the platform, software, and vulnerability detail needed for SUT construction?
RQ2: How far can those signals be translated into deployable SUT specifications, and where do they stop?
RQ3: How specific are SUT requirement profiles for campaigns and intrusion sets without over-claiming attribution?

Our contributions are threefold. First, we define a SUT-centric measurement model for environment evidence in structured CTI and apply it across the MITRE ATT&CK Enterprise, Mobile, and ICS datasets, with CAPEC and FiGHT as coverage contrasts. Second, we classify techniques by minimum backend requirement and measure how specific SUT profiles remain across campaigns and intrusion sets [18]. Third, we quantify the marginal gain of bounded enrichment and turn the resulting measurements into claim discipline: a reproducible distinction between environment narrowing that the corpus already supports and replay-ready derivation that still requires analyst-supplied detail.

The outcome is a reproducible baseline for judging current corpora, future schema changes, and new automation claims against the same environment-specification question.

2 Background

The MITRE ATT&CK framework documents adversary behavior as tactics, techniques, and sub-techniques across three domains: Enterprise, Mobile, and ICS. The Enterprise domain is the largest and most extensively documented, containing 835 techniques and sub-techniques across 14 tactics in v18.1 [23, 25]. ATT&CK content is distributed as STIX 2.x bundles [15]. For SUT construction, the most relevant object types are attack patterns, campaigns, intrusion sets, malware, tools, and the relationships among them. STIX captures semantic associations such as *uses*, *attributed-to*, and *targets*, but most ordering, environment bindings, and parameter flow still remain in natural-language `description` fields rather than machine-readable fields.

Table 1 summarizes the ATT&CK entity types most relevant to SUT construction.

Table 1: ATT&CK entity types and their SUT relevance.

Entity	SUT relevance
`attack-pattern`	Platform hints via `x_mitre_platforms`; system requirements in `x_mitre_system_requirements` (rarely populated).
`malware` / `tool`	Implies OS dependency; version rarely specified; CPE mappings largely absent.
`campaign`	Scope operation to the time window and target set; links to techniques and intrusion set.
`intrusion-set`	Aggregates TTP profile; useful for SUT susceptibility profiling but not for single-campaign instantiation.
`vulnerability`	CVE-mapped objects are present in some bundles; they are rarely linked directly to campaigns.

Red team platforms such as Caldera assume a compatible SUT already exists and do not derive environment configurations from CTI [1]. SUT construction therefore remains a manual precondition for emulation, which is exactly the burden this paper measures.

3 Related Work

Jin et al. show that CTI sharing provides limited operational value partly because shared feeds exhibit inconsistent field usage and low structural completeness [11]. Vermeer et al. measure gaps in STIX-encoded sources [24]. Prior ATT&CK surveys identify sparse software versioning as an open problem but do not quantify it [8, 23, 25]. This line of work establishes that structured CTI is unevenly populated, but it does not measure how that unevenness translates into deployable SUT evidence.

Provenance-graph APT detection systems reach a related conclusion from a different angle: operational coverage depends on what telemetry and ATT&CK-linked behaviors the underlying datasets can represent [7, 9, 10, 12, 14, 28, 30]. Those works measure *behavioral* completeness. They do not measure whether structured CTI contains enough information to instantiate the environment in which emulation is supposed to run.

A companion measurement study asks the procedural version of this question: whether ATT&CK-in-STIX encodes enough ordering, preconditions, and translation-ready detail for multi-step replay [31]. This paper isolates the environment version of the problem: whether the same corpora encode enough platform, software, vulnerability, and deployability evidence to build the SUT in which those steps could run.

Recent CTI extraction pipelines improve structured recovery of behavioral intelligence from reports, including TTP extraction and graph construction [2, 3, 19, 27]. Attribution-oriented work shows that analyst confidence is often bounded by evidence quality rather than by the absence of behavioral hypotheses [18, 29]. Both lines of work still prioritize *what* happened over *where* execution should occur, so they do not provide a measurement framework for SUT derivation quality.

Emulation and cyber-range systems assume that the environment has already been built. Tools such as Caldera [1], Atomic Red Team [17], Laccolith [16], AURORA [26], and recent LLM-guided proposals [5, 21, 22] provide execution logic or planning support over manually authored, fixed, or benchmarked lab environments, but they do not derive deployment-ready SUT specifications from CTI. Cyber range frameworks likewise rely on manually authored environment definitions that spell out OS images, service configuration, topology, and vulnerable software [4, 16]. The missing piece is a measurement of how much of that environment burden structured CTI already carries and how much still sits outside the schema.

4 SUT Measurement Model

Here, *environment* denotes the complete execution context required to reproduce the behavior described by a CTI object. This includes the operating system family and version, installed software (with version and configuration), network topology and reachability, identity and privilege model, and any exploitable vulnerability.

An environment is *machine-actionable* when every constituent constraint is expressed in a structured, parseable field such as x_

mitre_platforms, a Common Platform Enumeration (CPE) string, or a CVE identifier, rather than in free-text prose. The *System Under Test* (SUT) is the concrete instantiation of such an environment in a virtualized or containerized lab.

Each CTI object can contribute three kinds of SUT constraint. Platform constraints come from `x_mitre_platforms` and indicate an OS family, cloud provider, or device type. Software constraints arise when a `malware` or `tool` object is linked via `uses` relationships to a technique or campaign, implying that some installable artifact must exist in the SUT. Vulnerability constraints arise when a CVE identifier appears in descriptions or when a `vulnerability` object is linked to a campaign, technique, or software artifact.

For each campaign, and analogously for each intrusion set, we define a SUT requirement profile as the union of those three components. The profile is an analysis artifact rather than an execution plan. For reporting, we distinguish coarse profiles (software plus platform), anchored profiles (coarse plus version/CPE or campaign-level CVE), and exploit-pinned profiles (coarse plus campaign-level CVE).

Separately, we classify each technique by the minimum backend needed to satisfy its constraints: *container-feasible* for user-space behaviors without kernel, firmware, or Active Directory dependence; *VM-required* for kernel interaction, process injection, or boot-level persistence; and *infrastructure-dependent* for behaviors that presuppose multi-host topology, domain services, or enterprise identity.

The measurements derived from this model are intentionally simple. Platform coverage asks how often techniques carry x_mitre_platforms. Software reference rate asks how often campaigns and intrusion sets link to at least one malware or tool object. Vulnerability reference rate asks how often they carry CVE evidence. SUT profile specificity asks how often intrusion-set profiles remain unique under Jaccard distance threshold $\delta = 0.1$.

Table 2 summarizes how these signals should be read operationally.

A concrete example illustrates the tiers. A profile that links a platform family, one or more software artifacts, and campaign-level CVE evidence is exploit-pinned: it identifies a plausible vulnerable deployment family even if it still omits the exact lab topology. A profile with only platform and software evidence is coarse: it narrows the candidate deployment family but does not yet pin exploitability.

Table 2: Operational reading of structured SUT signals. The final column states what still remains outside current machine-actionable evidence.

Signal	What it narrows	What still remains manual
x_mitre_platforms	OS, device, or cloud family	exact version, host configuration, and deployment placement
Software link (`malware`/`tool`)	installable product family or compatibility envelope	version range, CPE, and configuration needed for installation
Campaign CVE evidence	exploit family and plausible vulnerable target class	exact vulnerable product-version tuple and trust boundary
Compatibility class (derived)	minimum backend family (CF/VMR/ID)	exact VM count, topology, domain layout, and service dependencies

Table 3: STIX object totals across ATT&CK domains and sources. Active counts after deprecation filtering are reported in Section 6.

Source	Techniques	Camp.	IS	Malware	Tools
Enterprise	835	52	187	696	91
Mobile	190	3	17	121	2
ICS	95	8	16	30	0
CAPEC	615	0	0	0	0
FiGHT	707	0	136	475	73

5 Dataset

Following the measurement model in Section 4, we select datasets that expose the same core STIX structures while covering different ATT&CK scopes. We use the MITRE ATT&CK Enterprise bundle v18.1 as the primary source for all quantitative analyses. This bundle is the most extensively documented domain.

Table 3 summarizes object counts across all analyzed sources.

The quantitative pipeline traverses all five local bundles for cross-corpus coverage metrics. RQ2 and RQ3 remain focused on Enterprise because only Enterprise contains campaign coverage sufficient for deployment-oriented analysis. CAPEC is included only as STIX-encoded attack-pattern knowledge for coverage contrast. Table 3 reports raw object totals; all RQ metrics use active-object counts after deprecation/revocation filtering.

The Enterprise bundle contains 20048 relationship objects. After deprecation and revocation filtering, our analysis covers 691 active techniques, 52 campaigns, 172 intrusion sets, and 784 software artifacts (693 malware + 91 tools).

From attack-pattern objects we extract x_mitre_platforms, x_mitre_system_requirements, x_mitre_permissions_required, and x_mitre_defense_bypassed. From malware and tool objects we extract names, aliases, platform tags, and any CPE or version strings in external_references. CVE identifiers are extracted from description fields and from linked vulnerability objects. Section 6 turns these fields into explicit measurement rules.

6 Measurement Methodology

This section turns the measurement model into explicit extraction and decision rules. We first describe how the bundles are parsed and normalized, then define the compatibility taxonomy used for deployability claims, then bound the enrichment steps we allow beyond structured fields, and finally specify the profile-specificity metric used in RQ3. The goal is to make every reported number traceable to a stable rule rather than to ad hoc interpretation.

6.1 Extraction Pipeline

We parse five locally available STIX bundles as JSON and normalize object identifiers. Enterprise, Mobile, and ICS drive the core analyses; CAPEC and FiGHT support cross-corpus coverage in RQ1. We exclude deprecated and revoked objects, which removes 144 of the 835 Enterprise attack-pattern objects, leaving 691 active techniques. Relationships are indexed by (*source_ref*, *relationship_type*, *target_ref*) triples to enable bidirectional traversal. For each campaign, the technique set is collected from all `attack-pattern` objects reachable via `uses` relationships; software and CVE associations are derived analogously.

For campaign-level target-platform inference, we use a conservative rule: platforms are inferred only from software-linked platform tags. Technique-level platforms are excluded from this step because they encode where a technique can run in general, not what a specific campaign targeted. Campaigns with no linked software are reported as *platform unknown* instead of receiving inferred values. This avoids turning generic compatibility into campaign-specific target claims.

Table 4: Compatibility classes as conservative lower-bound backend claims. The table states what each class asserts and which error direction it prefers when structured evidence is incomplete.

Class	Structured trigger	Operational reading and preferred conservative error
CF	Platforms restricted to Linux/Containers; no kernel, boot, or identity cues	Candidate for container-only replay. The rule is intentionally narrow, so the preferred error is false exclusion from CF rather than false CF optimism.
VMR	Privilege, kernel, firmware, or boot semantics; also the unresolved residue	Requires host-level VM semantics. The preferred error is false VMR inflation when structured fields are too weak to justify CF or ID.
ID	Domain, identity-provider, or Active Directory semantics; lateral movement with enterprise-service dependence	Requires enterprise services or multi-host structure. The class bounds backend family, not the exact topology, trust boundary, or host count.

6.2 SUT Compatibility Classification

Each technique receives exactly one compatibility class under a fixed precedence order. Identity and infrastructure cues dominate, host-semantic VM cues come next, and narrow container-feasible cues come last. Any remaining unresolved technique defaults to **VMR**.

More concretely, we assign **ID** when platform tags reference `Windows Domain` or `Identity Provider`, or when a lateral-movement technique depends on Active Directory software. We assign **VMR** when permissions-required fields indicate administrator privileges (Administrator, SYSTEM, or root), and likewise for privilege-escalation and boot/firmware defense-evasion techniques. We assign **CF** when `x_mitre_platforms` is restricted to `Containers` or `Linux` and the technique is not in kernel-interaction or boot-persistence clusters.

This taxonomy is a conservative deployability proxy rather than execution ground truth. Explicit rules cover 32.7% of techniques; the remaining 67.3% default to VMR so that unresolved cases overestimate rather than under-estimate infrastructure cost. CF is intentionally narrow, ID captures explicit enterprise-service dependence, and the unresolved middle goes to VMR because the safer failure mode for this paper is false VMR inflation rather than false CF optimism.

Table 4 makes the error budget explicit. If all unresolved techniques were optimistically forced to CF, the non-CF share would still be 29.9%; under VMR or ID defaults it rises to 97.2%. More importantly, the explicit-rule subset is already 91.6% non-CF. The fallback policy therefore changes the absolute split, but not the substantive conclusion that container-feasible techniques are a small minority and that the main residual uncertainty is upward VMR inflation rather than hidden CF overclaiming.

For independent auditing, the release package includes a stratified manual-validation packet of 36 techniques. It is balanced across CF/VMR/ID predictions and covers both explicit rules and fallback-driven VMR cases so that reviewers can inspect the narrow CF minority, the explicit ID evidence, and the conservative middle separately. The file is distributed as `results/audit/compatibility_validation_sample.csv`. At artifact release, the manual-label columns are intentionally blank, and the evaluator script computes agreement summaries only after a reviewer or shepherd fills them. The headline RQ2 percentages reported in this paper use deterministic rules only; external adjudication is provided as an audit mechanism and does not feed into the reported metrics.

6.3 Bounded Enrichment Rules

To quantify the practical impact of the measured gaps, we evaluate three bounded enrichment steps over the ATT&CK bundles used in this study: description-based regex mining for software versions, free-text extraction of campaign-level CVE references, and deterministic compatibility assignment with conservative fallback for unresolved techniques. By *headroom* we mean the additional fraction of environment signal that becomes machine-actionable after one such enrichment step is applied to the current corpus. Each step emits auditable CSV/JSON outputs. The resulting metrics quantify that recoverable margin without claiming full execution fidelity.

6.4 Profile Specificity Analysis

For each intrusion set g we encode its SUT profile as a binary vector over all software and CVE identifiers in the corpus. Pairwise Jaccard distances are computed and the fraction of groups with at least one neighbor within $\delta = 0.1$ is reported as the SUT profile confusion rate. Section 7 reports the measured values under these rules.

This metric is intentionally simple because the profiles themselves are sparse sets rather than weighted timelines. Jaccard distance is 0 for identical support and 1 for disjoint support. We use $\delta = 0.1$ as a near-neighbor threshold, so two profiles count as confusable only when at least 90% of their union is shared. Reporting the same rate under $k \geq 1$, $k \geq 2$, and $k \geq 3$ linked-software thresholds then separates genuine environmental overlap from ambiguity caused by extremely thin evidence.

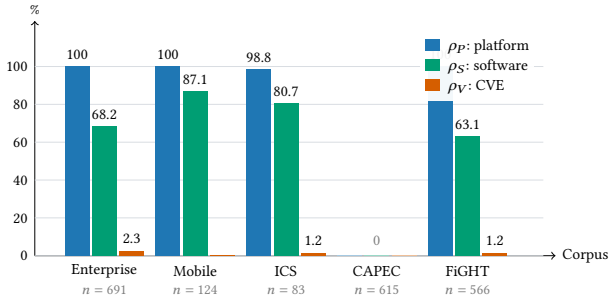

Figure 1: Cross-corpus coverage of structured SUT-relevant signals at attack-pattern level, with per-corpus denominators shown as n labels. Platform fields are near-universal, software-link coverage is moderate, and CVE-linked coverage is sparse across ATT&CK and related corpora. CAPEC zeros are schema-driven in this dataset.

The reproducibility package used in this study contains the measurement code, generated datasets, audit CSVs, and release checks. Its root-level verifier, `run_review_check.sh`, reruns extraction, rebuilds figures, regenerates traceability tables, and compiles the manuscript. Appendix A lists the anonymous artifact location used for review and the materials it exposes.

7 Analysis

We report results in RQ order: coverage (RQ1), deployability and compatibility (RQ2), and then profile specificity (RQ3).

7.1 Platform Constraint Coverage (RQ1)

Of the 691 active Enterprise techniques, all 691 (100.0%) carry at least one entry in `x_mitre_platforms`, so platform annotation is effectively mandatory in current ATT&CK curation. In contrast, `x_mitre_system_requirements`, the most specific platform field, is populated in 0 techniques (0.0%). Structured, fine-grained OS and software-version constraints are absent in the Enterprise environment.

Platform-field coverage is also high in Mobile (100.0%), ICS (98.8%), and FiGHT (100.0%) under the same active-object filter. CAPEC does not use `x_mitre_platforms` in this dataset because it encodes abstract attack patterns rather than platform-bound behaviors.

Across corpora, the same pattern appears: platform-field coverage is high, software-link coverage is moderate, and CVE-linked coverage is low. In CAPEC, the zeros are structural properties of the bundle: active `attack-pattern` objects link to no `malware` or `tool` objects and carry no CVE mentions. They are not missing data caused by the measurement pipeline. Figure 1 summarizes this cross-corpus coverage pattern directly.

Applying the software-only platform rule to all 52 Enterprise campaigns yields a platform signal for 47 campaigns (90.4%); the remaining 5 campaigns (9.6%) are genuinely unknown under structured fields alone. These campaigns are *FrostyGoop Incident*, *KV Botnet Activity*, *Leviathan Australian Intrusions*, *SPACEHOP Activity*, and *ShadowRay*.

Table 5 summarizes the OS-family distribution among campaigns with signal. These values are inferred from software-linked platform tags in ATT&CK, rather than from explicit campaign target declarations. They should be interpreted as evidence of compatibility. Campaigns may be multi-OS, so OS-family counts can exceed the number of campaigns with signal. When the platform signal is driven by a single cross-platform tool (e.g., Linux/Windows/macOS tags), the inferred set is a compatibility envelope rather than a confirmed target-OS declaration.

Table 5: Software-derived OS-family evidence across Enterprise campaigns. Values are multi-label inferences from software-linked platform tags and represent evidence of compatibility, not confirmed target-OS attribution. % signal uses $n = 47$; % total uses $n = 52$.

OS family	Campaigns	% signal	% total
Windows	42	89.4%	80.8%
Linux	23	48.9%	44.2%
macOS	21	44.7%	40.4%
Platform unknown	5	—	9.6%

How much that signal truly narrows the environment is mixed. Of the 44 campaigns with any OS-family evidence, 22 collapse to a single family, 2 remain dual-family, and 20 still expand to a Linux, Windows, and macOS envelope. Table 6 makes that split explicit and isolates the 17 campaigns whose OS-family inference is driven by exactly one linked software item. Even in that subset, 10 narrow cleanly while 7 still expand to a triple-family compatibility envelope.

3 additional campaigns carry only non-OS platform tags such as network-device labels and therefore do not narrow the OS family at all under the software-only rule. The platform signal is common, but it often remains broad.

OS-family evidence should therefore be read as compatibility evidence for SUT construction, not as a standalone attribution signal. The campaigns themselves are not behaviorally sparse: the 52 Enterprise campaigns use 19.6 techniques on average (median 17) across 8.0 ATT&CK tactic categories, and 44 campaigns (84.6%) exercise at least one technique in each of the initial-access, execution, discovery, lateral-movement, and exfiltration phases. Only 2 campaigns (3.8%) carry no explicit platform or software signal and rely entirely on tactic-level heuristics for environment inference.

Table 6: How software-derived platform signal actually narrows Enterprise campaigns. The first block uses campaigns with any OS-family signal ($n = 44$); the second isolates campaigns whose OS-family inference is driven by exactly one linked software item ($n = 17$).

Case	Campaigns
Single OS family	22
Two OS families	2
Linux, Windows, and macOS envelope	20
Single-OS, one-software case	10
Triple-OS, one-software case	7

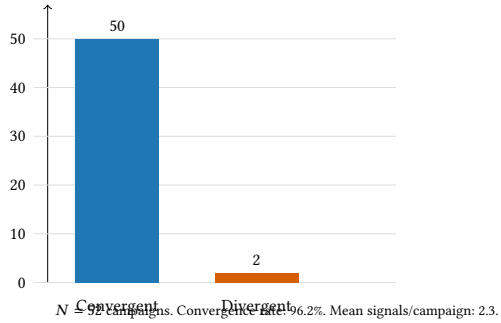

Figure 2: Cross-signal agreement for campaign platform inference in Enterprise ($N = 52$ campaigns). Three independent signal sources agree for 50 campaigns (96.2%), while the two divergent cases are preserved in the audit data rather than normalized away.

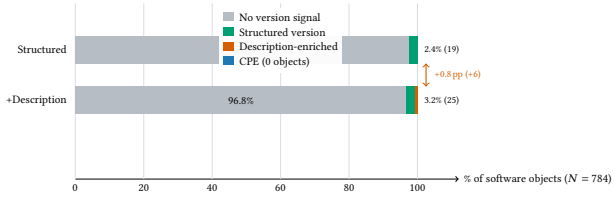

Figure 3: Most Enterprise software evidence is not version-pinned ($N = 784$ active software objects), limiting direct SUT instantiation. Only 2.4% of software objects carry any version signal, description enrichment adds 0.8 pp, and CPE coverage remains at zero.

To test whether this platform inference is internally stable rather than arbitrary, we compare three signal sources for each campaign: technique metadata, software-linked platforms, and tactic-implied heuristics. They converge for 96.2% of campaigns. The two divergent cases are *Operation MidnightEclipse* and *RedPenguin*, where technique-level platforms define a broad cross-platform compatibility envelope while software-linked evidence points to Linux and tactic-implied heuristics point to Windows. We keep those conflicts explicit in the audit package rather than collapsing them into a single OS claim.

7.2 Software Reference Rate (RQ1, RQ2)

The Enterprise bundle contains 693 active malware and 91 tool objects (784 total). Of the 52 campaigns, 47 (90.4%) are linked to at least one software artifact. Among the 172 active intrusion sets, 158 (91.9%) carry at least one software association.

Specificity is the limiting factor. Only 2.4% of the 784 software objects contain any version indicator, whether in `name`, `aliases`, or `external_references`. CPE identifiers, which directly support version-pinned installation, appear in only 0.0% of software objects. Software *presence* is well documented, but software *specificity* for automated SUT construction is largely absent. Figure 3 shows the consequence directly: most software references remain family-level labels without a version pin.

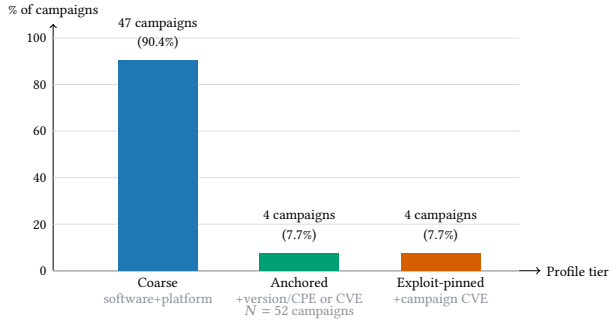

Figure 4: Campaign SUT profile completeness collapses from coarse to anchored/exploit-pinned levels in Enterprise ($N = 52$ campaigns). Linkage abundance does not imply version-pinned SUT precision.

At campaign level, that bottleneck propagates into profile completeness: coarse profiles are common, while anchored and exploit-pinned profiles are rare. Figure 4 shows this collapse directly.

In practice, structured fields already support coarse environment profiles for 47 campaigns, which is useful for initial narrowing before analyst refinement. Version-anchored precision remains rare. We therefore next ask how often vulnerability evidence can supply that missing anchor.

7.3 Vulnerability Reference Rate (RQ1, RQ2)

CVE identifiers extracted from the Enterprise bundle total 26 unique CVEs. The bundle contains zero dedicated vulnerability SDOs, so CVE evidence appears in structured external references or free-text descriptions. Of the 26 CVEs, 14 appear only as illustrative examples in technique descriptions (for example, *Exploit Public-Facing Application* citing CVE-2016-6662). These are not campaign-specific. The remaining 12 are actionable because they are linked to software, campaign, or intrusion-set objects.

Coverage remains low at campaign and group level. Only 5 campaigns (9.6%) carry any CVE reference, and only 4 intrusion sets (2.3%) have at least one CVE. This is lower than the 31.6% ATT&CK-group CVE coverage reported by Saha et al., who used full threat-report parsing rather than structured STIX fields alone [18]. The gap is therefore not only in public reporting, but in what becomes machine-readable. Even when vulnerability evidence exists, it is split across structured external references and free-text descriptions.

To test how much of this gap is recoverable without changing the corpus, we promote CVE mentions from free-text descriptions into machine-readable CVE slots and then recount coverage. Under this normalization, campaign-level CVE coverage rises from 2/52 (3.8%) to 5/52 (9.6%), a gain of 3 campaigns (+5.8 percentage points). For intrusion sets, coverage rises from 2/172 (1.2%) to 4/172 (2.3%), a gain of 2 (+1.1 percentage points). Figure 5 summarizes that attrition path from all detected CVEs to campaign-level usable evidence.

The gain remains modest because most campaigns do not provide explicit vulnerability evidence in any form. Only 8 actionable CVEs are tied directly to campaign objects in the Enterprise STIX

dataset. Table 7 lists those CVEs and the five campaigns they belong to; they represent the strongest structured vulnerability evidence for SUT instantiation in the current bundle.

Table 7: Campaign-linked actionable CVEs in Enterprise ATT&CK v18.1. Each CVE is extracted from the structured `external_references` field of a campaign object.

Campaign	CVEs
APT28 Nearest Neighbor Campaign	CVE-2022-38028
ShadowRay	CVE-2023-48022
Operation MidnightEclipse	CVE-2024-3400
Versa Director Zero Day Exploitation	CVE-2024-39717
SharePoint ToolShell Exploitation	CVE-2025-49704; CVE-2025-49706; CVE-2025-53770; CVE-2025-53771

Campaign-level CVE evidence is therefore necessary but not sufficient for an anchored profile. Table 8 shows the five campaigns with any CVE signal. Four also retain linked software and platform evidence and therefore reach the exploit-pinned tier. *ShadowRay* is the counterexample: it carries CVE-2023-48022, but with no linked software object and no software-derived platform signal it still does not anchor a deployable SUT profile under the conservative rules used in this paper.

Table 8: Outcomes for the five Enterprise campaigns with any CVE signal. CVE presence alone does not guarantee an anchored SUT profile.

Campaign	SW links	Plat.	Tier
APT28 Nearest Neighbor Campaign	Yes (2)	Yes	Exploit-pinned
Operation MidnightEclipse	Yes (1)	Yes	Exploit-pinned
Versa Director Zero Day Exploitation	Yes (1)	Yes	Exploit-pinned
SharePoint ToolShell Exploitation	Yes (4)	Yes	Exploit-pinned
ShadowRay	No (0)	No	Not anchored

These entries are from the ATT&CK v18.1 bundle snapshot used in this study, extracted on March 5, 2026; CVE year values reflect assignment year, not bundle publication year.

The contrast between *SharePoint ToolShell Exploitation* and *ShadowRay* makes the boundary concrete. *SharePoint ToolShell Exploitation* reaches the exploit-pinned tier because the bundle carries four campaign-linked CVEs, linked software, and a software-derived platform signal. Even in that strongest case, the structured evidence still stops short of a replay-ready SUT: ATT&CK names vulnerable software and CVEs, but not the exact product-version tuple, trust boundary, or deployment topology needed to instantiate the lab automatically.

ShadowRay fails in the opposite direction. It carries a campaign-level CVE, but without linked software and without a software-derived platform signal that vulnerability reference cannot be turned into a deployable SUT profile. CVE evidence can pin exploitability only after software identity and coarse platform have already been established.

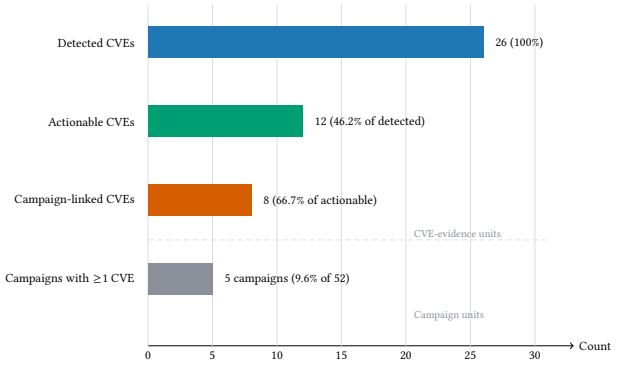

Figure 5: Operational CVE funnel in Enterprise STIX data, where detected CVEs (26) narrow to actionable CVEs (12), then to campaign-linked CVEs (8), and finally to campaigns with any CVE signal (5 of 52). The final stage is a deployment endpoint in campaign units, while prior stages are CVE-evidence units.

38/52 campaigns (73.1%) are linked to at least one Initial Access technique even though CVE evidence remains sparse. Of those, 33 campaigns (63.5%) lack campaign-level CVE evidence, confirming that Initial Access is often documented without explicit exploit identifiers in the current encoding.

7.4 SUT Compatibility Classification (RQ2)

RQ2 asks whether the environment signal measured above is strong enough to support backend claims. Our answer is conservative: CF/VMR/ID is a lower-bound backend taxonomy rather than execution ground truth. Among the 691 active Enterprise techniques, 19 (2.7%) are container-feasible, 526 (76.1%) are VM-required, and 146 (21.1%) are infrastructure-dependent.

Figure 6 combines class distribution with a non-CF sensitivity interval, including fallback floor and ceiling and the rule-only resolved estimate, so the chart adds uncertainty context beyond the raw class counts.

Techniques classified as VMR or ID cannot be reproduced faithfully in a container-only environment. Campaigns involving lateral movement, boot-level persistence, or credential access through domain services consistently fall into VMR or ID classes, which sharply limits the coverage of container-based cyber ranges.

The conservative fallback materially affects absolute rates. When unresolved techniques are forced to CF, VMR, or ID, the non-CF share spans 29.9% to 97.2%. Without fallback assignment, explicit rules cover 32.7% of techniques, and the resolved subset is 91.6% in the non-CF class. Under every sensitivity setting, non-CF techniques remain the majority, so container-only replay covers only a small minority of ATT&CK. In Figure 6, the right-side interval is therefore a policy-sensitivity band, not a confidence interval: the floor forces unresolved techniques to CF, the ceiling forces them to ID, and the marker reports the explicit-rule subset before fallback completion.

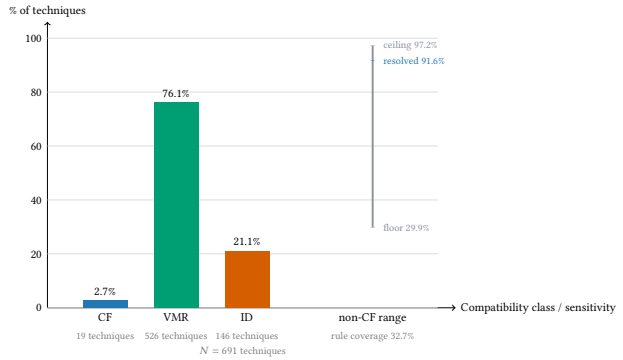

Figure 6: Compatibility-class distribution for active Enterprise techniques. Most techniques are VM-required (VMR) or infrastructure-dependent (ID), bounding the coverage of container-only emulation setups. The right-side interval summarizes non-CF sensitivity under fallback choices, and the marker reports the rule-only resolved estimate.

All 691 techniques receive a deterministic class assignment (CF, VMR, ID). This supports automated backend selection even when faithful replay is not supported. Table 9 exposes where those assignments come from and shows that the strongest part of the taxonomy is a lower bound on container-feasible coverage, while the weaker part is conservative VMR completion. A tactic-level decomposition is retained in Appendix D for reviewers who want the per-tactic split, but it does not change the result.

Table 9: Explicit-rule breakdown for Enterprise compatibility classification. The table makes the evidence-bearing minority and the conservative fallback majority explicit.

Rule evidence	Assigned class	Techniques	Share
Platform keyword (`Windows Domain/identity/IaaS/SaaS`)	ID	132	19.1%
Lateral-movement tactic fallback	ID	14	2.0%
Kernel or boot pattern	VMR	34	4.9%
Privileged or kernel name pattern	VMR	27	3.9%
Container-compatible targets	CF	19	2.7%
Conservative fallback	VMR	465	67.3%

Table 9 exposes the rule surface directly. Most ID assignments come from explicit platform cues already present in ATT&CK, while most VMR assignments come from conservative fallback rather than from a rich machine-readable notion of hypervisor or domain topology. The taxonomy is therefore strongest as a lower bound on container-only coverage: it reliably identifies the small CF minority and the broad non-CF majority, but it does not claim that structured CTI fully specifies the exact VM layout or multi-host topology required within that majority.

The compatibility analysis tells us what kind of lab a technique needs. The next question is whether those observable SUT properties can distinguish one threat actor from another (RQ3).

7.5 SUT Profile Specificity (RQ3)

Of the 172 active intrusion sets, 90.7% have a unique SUT profile at Jaccard distance $\delta = 0.1$ when profiles use software constraints only. Adding CVE constraints leaves this value unchanged at 90.7%. That result is consistent with sparse CVE linkage: only 4 intrusion sets carry any CVE evidence. The remaining 9.3% of groups still share their SUT profile with at least one other group.

A SUT built for group g can therefore still match profiles of other groups within distance δ, so a successful emulation is not actor-specific by default. The same ambiguity pattern observed in behavioral attribution studies [18] reappears here at the environmental level. Bootstrap resampling with 5,000 replicates leaves that reading unchanged: confusion remains within 5.2%–14.0%, and the unique-profile rate remains within 86.0%–94.8%. The reading is also flat across the tested near-neighbor thresholds: confusion stays at 9.3% for $\delta \in \{0.05, 0.10, 0.15, 0.20, 0.30\}$.

The ablations point in the same direction. Adding CVE, platform, or OS-family signal does not materially improve separability beyond software identity alone. Compatibility-summary features help only modestly, improving confusion by 0.6 pp. A cardinality-preserving null model (1000 iterations, randomizing software identities while keeping per-IS counts) places the observed confusion inside its expected interval (8.1%–10.5%, permutation $p = 0.34$). The key result is that confusion is largely a low-evidence phenomenon: at $\delta = 0.10$, it disappears once a group carries at least two linked software items.

Table 10: RQ3 feature ablation at $\delta = 0.10$ (n = 172 intrusion sets).

Profile Setting	Unique (%)	Confused (%)
Software only	90.7%	9.3%
Software + CVE	90.7%	9.3%
Software + platform	90.7%	9.3%
Software + CVE + platform	90.7%	9.3%
Software + OS family	90.7%	9.3%
Software + compatibility	**91.3%**	**8.7%**

Figure 7 should be read as an evidence-threshold plot: the x-axis raises the minimum number of linked software items required for a group to enter the comparison set, and the y-axis reports the share of groups that still have a near neighbor under $\delta = 0.10$. Figure 8 then provides the full nearest-neighbor CDF comparison. In that plot, a curve farther right would indicate better separation, so the near overlap of the software-only and software+CVE curves confirms that adding CVE evidence does not materially shift separability at $\delta = 0.10$. Appendix D points reviewers to the corresponding audit files.

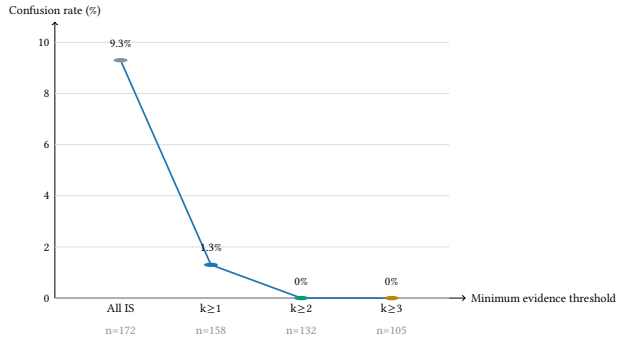

Figure 7: Profile ambiguity collapses by $k \geq 2$ linked software items in software-only profiles ($\delta = 0.10$) and remains collapsed for higher thresholds.

7.6 Bounded Enrichment Gains

We now quantify the marginal value of bounded enrichment steps already implemented in the release package. This subsection reports how much additional environment signal becomes machine-actionable when description mining and deterministic compatibility assignment are added on top of structured ATT&CK fields.

Structured ATT&CK fields expose version signal for only 2.4% of active software objects. Description-based regex enrichment raises this to 3.2%, a gain of 0.8 pp. Campaign-level CVE coverage shows a similarly bounded pattern: structured links alone cover 3.8% of campaigns, and free-text CVE extraction raises this to 9.6% (+5.8 pp). Compatibility assignment behaves differently. Explicit rules cover 32.7% of Enterprise techniques, while a deterministic fallback policy resolves 91.6% of non-CF techniques, yielding a larger 58.9 pp increase in backend assignment coverage.

Table 11 places those three margins side by side so reviewers can compare what bounded enrichment actually recovers and what it still leaves unresolved.

Table 11: Measured headroom from bounded enrichment steps.

Measure	Base	Enriched	Gain
Software version signal	2.4%	3.2%	+0.8
Campaigns with ≥1 CVE	3.8%	9.6%	+5.8
Compatibility assignment	32.7%	91.6%	+58.9

The gains are uneven. Version signal rises by 0.8 pp, campaign-level CVE coverage by 5.8 pp, and backend assignment by 58.9 pp. The largest measured gain therefore comes from deterministic compatibility assignment rather than from version or CVE extraction.

8 From CTI Evidence to Partial SUT Design

Section 7 measures the automation boundary at corpus scale. This section asks the corresponding operational question: *if I start from the structured CTI bundle in front of me, what kind of SUT can I derive before analyst specification begins?* We answer it by organizing the measured layers into one evidentiary stack and then tracing representative campaigns through that stack.

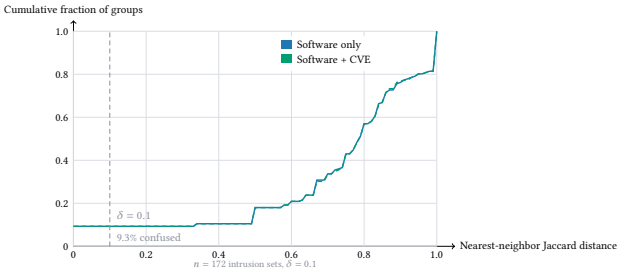

Figure 8: Nearest-neighbor profile confusion CDF for intrusion-set SUT profiles under software-only and software+CVE constraints.

Table 12 is the structural lesson of the paper. The early rows ask whether CTI can narrow the candidate environment, the middle rows ask whether that environment is deployable on a container, VM, or enterprise backend, and the last rows ask whether the resulting profile is distinctive and how much downstream enrichment can recover.

That layered view becomes concrete when we follow individual campaigns rather than aggregate rates alone. Figure 9 traces three campaigns that occupy different positions in the stack. It should be read from left to right: structured CTI first narrows the environment, the pipeline then derives the strongest SUT claim justified by those fields, and the first missing layer marks where analyst specification resumes.

The contrast between these rows is the operational core of the paper. *SharePoint ToolShell Exploitation* is the strongest structured case in the Enterprise corpus and still stops short of replay-ready derivation. *Operation MidnightEclipse* shows exploit pinning on thin evidence, while *ShadowRay* shows that a CVE alone cannot anchor a SUT without supporting software and platform evidence.

Operationally, the derivation path behaves like a monotone filter rather than a generator. Platform tags rule out impossible environment families, software links add installable artifacts, campaign-linked CVEs pin exploitability only relative to those artifacts, and compatibility classes choose the minimum backend family. The first missing layer is where analyst specification begins, which the artifact exposes step by step.

Table 13 turns the measurement into claim discipline. It separates useful narrowing from faithful instantiation and makes explicit that the current corpus does not determine an installable, actor-specific, replay-ready lab without additional analyst specification. Read operationally, it also functions as a reviewer-facing protocol for CTI-driven emulation claims: which claims are supported by current structured evidence, which are only conditionally justified, and which should still be treated as analyst-supplied.

9 Discussion and Implications

The measurements locate a stable limit in current structured CTI. Platform tags and software links usually narrow candidate environments, but anchored software and vulnerability evidence remain rare. Sections 7 and 8 show that bounded enrichment adds only modest margins, so most of the remaining analyst burden is a corpus problem rather than a parsing problem.

Table 12: How the measured evidence stack supports partial SUT design. Each row contributes a distinct part of the automation boundary, so the rows should be read cumulatively rather than as restatements of the same claim.

Evidence layer	Measured signal	What structured CTI already supports	What still remains outside the current corpus
Platform coverage	Near-universal platform tags; sparse system-requirement fields	Broad OS, device, or cloud-family narrowing and software-derived compatibility envelopes	Exact OS version, host configuration, deployment placement, and most campaign-specific target declarations
Software specificity	Campaign and intrusion-set software links are common, but version and CPE signal are rare	Coarse profiles and installable product-family hints; occasional anchored profiles when version/CPE appears	Product-version tuples precise enough for unattended installation and replay-ready package selection
Vulnerability evidence	Few campaign-linked CVEs; many CVEs remain only in prose	Exploit pinning for a small minority of campaigns when CVE evidence aligns with software and platform signal	Machine-readable exploit surface for most campaigns and direct links from vulnerabilities to deployable targets
Compatibility classification	CF/VMR/ID backend classes for every Enterprise technique	Deterministic backend-family selection and a lower bound on container-only coverage	Exact VM count, domain layout, multi-host routing, and service dependencies within the non-CF majority
Profile specificity	Nearest-neighbor confusion over intrusion-set SUT profiles	A test of whether derived environments are distinctive enough for evidence-filtered actor screening	Actor-specific interpretation under sparse evidence; environmental overlap remains common until software density rises
Bounded enrichment	Description mining and deterministic fallback assignment	A reproducible lower bound on recoverable signal without changing the underlying corpus	Closure of the gap through downstream extraction alone; the measured gains remain modest

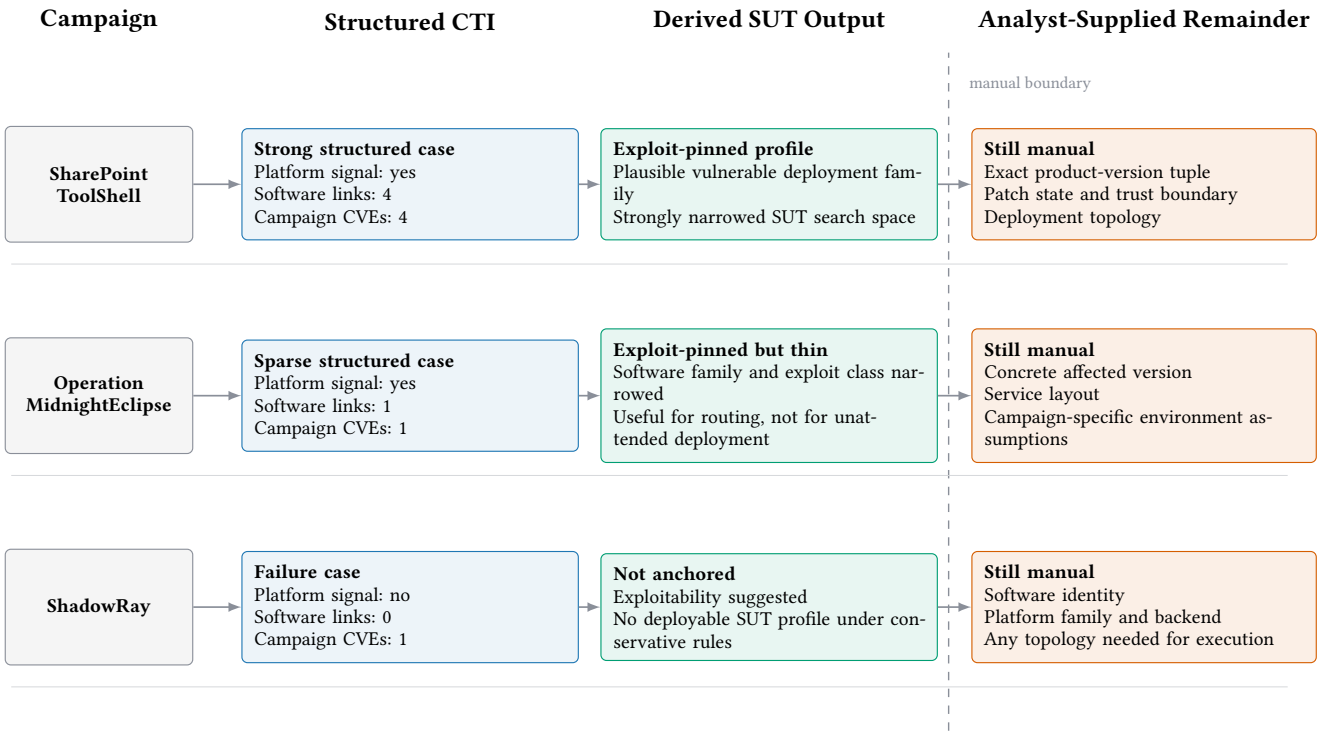

Figure 9: Worked examples showing how structured evidence translates into partial SUT design. Each row starts with the structured evidence already present in the bundle, then shows the strongest SUT claim the conservative pipeline can derive, and finally makes the analyst-supplied remainder explicit. The dashed line marks the automation boundary.

Table 13: What the measured evidence stack does and does not license reviewers to claim about SUT design.

Claim type	Status	Why the paper supports or rejects it
Coarse environment narrowing	Supported	Near-universal platform tags and common software links narrow OS, device, or product family even when exact versions remain missing.
Backend-family routing	Supported	CF/VMR/ID yields deterministic container-vs-VM-vs-infrastructure routing, and the non-CF majority persists under every fallback policy we report.
Exploit-pinned screening	Qualified	Campaign CVE evidence can pin exploitability for a small minority of cases, but only when software and platform evidence already align.
Actor-specific emulation	Conditional	The paper supports it only with an explicit confusion threshold and enough software density; sparse profiles still overlap across groups.
Replay-ready unattended SUT derivation	Not supported	Version, CPE, topology, service layout, and trust-boundary details remain outside current structured fields.

That boundary still supports useful automation. Structured CTI already supports coarse environment narrowing, deterministic backend selection via CF/VMR/ID, and evidence-filtered profile screening once profiles contain enough software signal. It also bounds what a cyber range can realistically cover: because 76.1% (VMR) plus 21.1% (ID) of Enterprise techniques require VM or full-infrastructure backends, environments built entirely around containers will miss boot-level persistence, kernel interaction, and enterprise identity attacks. The exact percentage shifts with fallback policy, but the dominance of non-CF techniques does not.

Profile analysis sharpens a second point. A SUT that matches the software and CVE profile of one group can still match several others, so claims of "APT-X-specific emulation" need an explicit profile-confusion threshold. In this corpus, confusion is 1.3% at $k \geq 1$ and falls to 0.0% at $k \geq 2$ linked software items. The same reading survives the δ sensitivity check.

The missing semantics are concrete rather than abstract. Most campaign-linked software still lacks version information and CPE identifiers; 5 of 26 CVEs (19.2%) appear only in natural-language `description` fields; and the schema carries no field that directly states whether a technique requires a container, a VM, or a multi-host enterprise topology. Bounded enrichment quantifies the upside of better curation, but its gains remain modest. Upstream structured curation therefore matters more than downstream text mining.

Table 14 summarizes four small extensions with measured impact on automated SUT generation:

Table 14: Proposed STIX schema extensions and measured lower-bound impact on deployability.

Extension	Current coverage	Lower-bound gain
`x_mitre_version_range`	2.4%	+0.8 pp
`cpe23uri` (software)	0.0%	+0.8 pp
`x_mitre_sut_class`	32.7% explicit	+67.3 pp
`structured_cve_links`	3.8% campaigns	+5.8 pp

These extensions are intentionally small rather than a schema redesign. x_mitre_version_range would represent installable version intervals without requiring exact versions. x_mitre_sut_class would encode CF/VMR/ID directly and remove heuristic backend assignment. Structured CVE links would move vulnerability evidence out of prose and into machine-readable relationships. The identical +0.8 pp gains for x_mitre_version_range and cpe23uri are expected because the same 6 recovered software objects define both margins. Table 14 therefore reports conservative lower bounds that reviewers can recompute from the released data.

Read together, Tables 13 and 14 turn a negative result into a tractable design target. Current corpora already support candidate-environment narrowing, backend-family routing, and selective exploit screening, then fail at a specific point: replay-ready deployment still needs version anchoring, explicit vulnerability structure, and declared backend semantics. Future CTI schemas and corpora can now be judged against a measurable question: do they shrink the analyst-supplied remainder exposed in Section 8, or do they merely restate behavior in richer prose?

10 Threats to Validity

Construct validity is limited because our measurements capture only what is explicitly encoded in STIX fields. Version and configuration information present in source reports but absent from structured fields is therefore out of scope. Internal validity is constrained by the asynchronous update pipeline between the ATT&CK website and downloadable STIX bundles, which may cause our extracted counts to underestimate true association rates. External validity is bounded by the focus on public ATT&CK datasets; commercial CTI feeds may carry richer software version data.

Campaign-level platform inference also uses software-linked platform tags as a proxy for compatibility. Multi-platform tools can inflate OS-family counts and do not prove the exact operating system targeted in a specific incident.

The compatibility taxonomy introduces a second construct-validity risk: some Linux-only techniques may still require richer host semantics than the structured fields reveal. We mitigate that risk by keeping CF intentionally narrow, defaulting unresolved cases upward to VMR, and publishing both explicit-rule coverage and fallback sensitivity so reviewers can see how much of the distribution is rule-driven versus conservative completion.

The platform-inference convergence check summarized in Figure 2 is released at campaign granularity in the artifact (results/audit/evidence_convergence.csv) for independent verification.

11 Conclusion

This paper turns a familiar operational complaint into a measured boundary in current ATT&CK-style structured CTI: how far environment evidence travels before analyst specification resumes.

Across five corpora, platform annotations are abundant, but version-anchored software and campaign-linked vulnerability evidence remain rare.

That result changes what practitioners can now claim. Structured CTI already supports candidate-environment narrowing, backend-family routing, and selective exploit screening. What it still does not support is unattended derivation of a replay-ready SUT, so anything beyond that boundary should be understood as analyst-supplied or tool-specific rather than corpus-supported.

That boundary now gives the community a measurable target. Future corpora and schema revisions can be judged by whether they shrink the analyst-supplied remainder through version anchoring, structured CVE links, and explicit compatibility semantics.

Acknowledgments

This work was supported in part by the Coordenação de Aperfeiçoamento de Pessoal de Nível Superior (CAPES), National Council for Scientific and Technological Development (CNPq), and by the Programa de Pós-Graduação em Aplicações Operacionais (PPGAO), Aeronautics Institute of Technology (Instituto Tecnológico de Aeronáutica – ITA). It also received support from the São Paulo Research Foundation (FAPESP), Grants #2020/09850-0 and #2022/00741-0. We thank the C2-DC Laboratory for providing the experimental infrastructure.

References

[1] Andy Applebaum, Doug Miller, Blake Strom, Chris Korban, and Ross Wolf. 2016. Intelligent, Automated Red Team Emulation. In *Proceedings of the 2016 ACM Workshop on Artificial Intelligence and Security*. Association for Computing Machinery, New York, NY, USA, 7–14. doi:10.1145/2996758.2996762

[2] Marvin Büchel, Tommaso Paladini, Stefano Longari, Michele Carminati, Stefano Zanero, Hodaya Binyamini, Gal Engelberg, Dan Klein, Giancarlo Guizzardi, Marco Caselli, Andrea Continella, Maarten van Steen, Andreas Peter, and Thijs van Ede. 2025. SoK: Automated TTP Extraction from CTI Reports - Are We There Yet?. In *34th USENIX Security Symposium, USENIX Security 2025, Seattle, WA, USA, August 13-15, 2025*, Lujo Bauer and Giancarlo Pellegrino (Eds.). USENIX Association, Seattle, WA, USA, 4621–4641. https://www.usenix.org/conference/usenixsecurity25/presentation/buechel

[3] Yutong Cheng, Osama Bajaber, Saimon Amanuel Tsegai, Dawn Song, and Peng Gao. 2025. CTINexus: Automatic Cyber Threat Intelligence Knowledge Graph Construction Using Large Language Models. In *Proceedings of the 2025 IEEE 10th European Symposium on Security and Privacy (EuroS&P)*. IEEE, Dublin, Ireland, 923–938. doi:10.1109/EuroSP63326.2025.00057

[4] Suresh K. Damodaran and Paul D. Rowe. 2025. Automated Repeatable Adversary Threat Emulation with Effects Language (EL). arXiv:2510.06420 arXiv:2510.06420.

[5] Gelei Deng, Yi Liu, Víctor Mayoral-Vilches, Peng Liu, Yuekang Li, Yuan Xu, Tianwei Zhang, Yang Liu, Martin Pinzger, and Stefan Rass. 2024. PENTESTGPT: Evaluating and Harnessing Large Language Models for Automated Penetration Testing. In *Proceedings of the 33rd USENIX Security Symposium* (Philadelphia, PA, USA) *(SEC '24)*. USENIX Association, USA, Article 48, 18 pages.

[6] Feng Dong, Shaofei Li, Peng Jiang, Ding Li, Haoyu Wang, Liangyi Huang, Xusheng Xiao, Jiedong Chen, Xiapu Luo, Yao Guo, and Xiangqun Chen. 2023. Are we there yet? An Industrial Viewpoint on Provenance-based Endpoint Detection and Response Tools. In *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security, CCS 2023, Copenhagen, Denmark, November 26-30, 2023*, Weizhi Meng, Christian Damsgaard Jensen, Cas Cremers, and Engin Kirda (Eds.). ACM, Copenhagen, Denmark, 2396–2410. doi:10.1145/3576915.3616580

[7] Feng Dong, Liu Wang, Xu Nie, Fei Shao, Haoyu Wang, Ding Li, Xiapu Luo, and Xusheng Xiao. 2023. DISTDET: A Cost-Effective Distributed Cyber Threat Detection System. In *32nd USENIX Security Symposium, USENIX Security 2023, Anaheim, CA, USA, August 9-11, 2023*, Joseph A. Calandrino and Carmela Troncoso (Eds.). USENIX Association, Anaheim, CA, USA, 6575–6592. https://www.usenix.org/conference/usenixsecurity23/presentation/dong-feng

[8] Keisuke Furumoto, Tomohiro Morikawa, Antti Kolehmainen, Bilhanan Silverajan, Takeshi Takahashi, and Daisuke Inoue. 2025. A Comprehensive Survey of Threat Intelligence Research: A Measurement-Based Study. *ACM Comput. Surv.* 58, 6, Article 153 (2025), 35 pages. doi:10.1145/3772280

[9] Xueyuan Han, Thomas Pasquier, Adam Bates, James W. Mickens, and Margo I. Seltzer. 2020. UNICORN: Runtime Provenance-Based Detector for Advanced Persistent Threats. arXiv preprint arXiv:2001.01525. https://arxiv.org/abs/2001.01525 Unpublished.

[10] Zian Jia, Yun Xiong, Yuhong Nan, Yao Zhang, Jinjing Zhao, and Mi Wen. 2024. MAGIC: Detecting Advanced Persistent Threats via Masked Graph Representation Learning. In *33rd USENIX Security Symposium, USENIX Security 2024, Philadelphia, PA, USA, August 14-16, 2024*, Davide Balzarotti and Wenyuan Xu (Eds.). USENIX Association, Philadelphia, PA, USA, 19 pages. https://www.usenix.org/conference/usenixsecurity24/presentation/jia-zian

[11] Beomjin Jin, Eunsoo Kim, Hyunwoo Lee, Elisa Bertino, Doowon Kim, and Hyoungshick Kim. 2024. Sharing cyber threat intelligence: Does it really help?. In *31st Annual Network and Distributed System Security Symposium, NDSS 2024, San Diego, California, USA, February 26 - March 1, 2024*. The Internet Society, San Diego, CA, USA, 20 pages. https://www.ndss-symposium.org/ndss-paper/sharing-cyber-threat-intelligence-does-it-really-help/

[12] Shaofei Li, Feng Dong, Xusheng Xiao, Haoyu Wang, Fei Shao, Jiedong Chen, Yao Guo, Xiangqun Chen, and Ding Li. 2024. NODLINK: An Online System for Fine-Grained APT Attack Detection and Investigation. In *Proceedings 2024 Network and Distributed System Security Symposium (NDSS 2024)*. Internet Society, San Diego, CA, USA, 18 pages. doi:10.14722/ndss.2024.23204

[13] Mingqi Lv, Hongzhe Gao, Xuebo Qiu, Tieming Chen, Tiantian Zhu, Jinyin Chen, and Shouling Ji. 2024. TREC: APT Tactic / Technique Recognition via Few-Shot Provenance Subgraph Learning. In *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security, CCS 2024, Salt Lake City, UT, USA, October 14-18, 2024*, Bo Luo, Xiaojing Liao, Jun Xu, Engin Kirda, and David Lie (Eds.). ACM, Salt Lake City, UT, USA, 139–152. doi:10.1145/3658644.3690221

[14] Sadegh M. Milajerdi, Rigel Gjomemo, Birhanu Eshete, R. Sekar, and V. N. Venkatakrishnan. 2019. HOLMES: Real-Time APT Detection through Correlation of Suspicious Information Flows. In *2019 IEEE Symposium on Security and Privacy (SP)*. IEEE, Los Alamitos, CA, USA, 1137–1152. doi:10.1109/SP.2019.00026

[15] OASIS Cyber Threat Intelligence (CTI) TC. 2021. *STIX Version 2.1*. OASIS Committee Specification 02. OASIS Open. https://docs.oasis-open.org/cti/stix/v2.1/cs02/stix-v2.1-cs02.html Latest stage: https://docs.oasis-open.org/cti/stix/v2.1/stix-v2.1.html.

[16] Vittorio Orbinato, Marco Carlo Feliciano, Domenico Cotroneo, and Roberto Natella. 2024. Laccolith: Hypervisor-based Adversary Emulation with Anti-detection. *IEEE Transactions on Dependable and Secure Computing* 21, 6 (2024), 5374–5387.

[17] Red Canary. 2023. Atomic Red Team. Software repository, GitHub. https://github.com/redcanaryco/atomic-red-team Accessed: 2026-03-05.

[18] Aakanksha Saha, James Mattei, Jorge Blasco, Lorenzo Cavallaro, Daniel Votipka, and Martina Lindorfer. 2025. Expert Insights into Advanced Persistent Threats: Analysis, Attribution, and Challenges. In *34th USENIX Security Symposium, USENIX Security 2025, Seattle, WA, USA, August 13-15, 2025*, Lujo Bauer and Giancarlo Pellegrino (Eds.). USENIX Association, Seattle, WA, USA, 2185–2204. https://www.usenix.org/conference/usenixsecurity25/presentation/saha

[19] Kiavash Satvat, Rigel Gjomemo, and V. N. Venkatakrishnan. 2021. Extractor: Extracting Attack Behavior from Threat Reports. In *2021 IEEE European Symposium on Security and Privacy (EuroS&P)*. IEEE, Los Alamitos, CA, USA, 598–615. doi:10.1109/EuroSP51992.2021.00046

[20] Xiangmin Shen, Zhenyuan Li, Graham Burleigh, Lingzhi Wang, and Yan Chen. 2024. Decoding the MITRE Engenuity ATT&CK Enterprise Evaluation: An Analysis of EDR Performance in Real-World Environments. In *Proceedings of the 19th ACM Asia Conference on Computer and Communications Security* (Singapore, Singapore) *(ASIA CCS '24)*. Association for Computing Machinery, New York, NY, USA, 96–111. doi:10.1145/3634737.3645012

[21] Xiangmin Shen, Lingzhi Wang, Zhenyuan Li, Yan Chen, Wencheng Zhao, Dawei Sun, Jiashui Wang, and Wei Ruan. 2025. PentestAgent: Incorporating LLM Agents to Automated Penetration Testing. In *Proceedings of the 20th ACM Asia Conference on Computer and Communications Security (ASIA CCS '25)*. Association for Computing Machinery, New York, NY, USA, 375–391. doi:10.1145/3708821.3733882

[22] Brian Singer, Keane Lucas, Lakshmi Adiga, Meghna Jain, Lujo Bauer, and Vyas Sekar. 2025. InCALMO: An Autonomous LLM-assisted System for Red Teaming Multi-Host Networks. arXiv preprint arXiv:2501.16466. https://arxiv.org/abs/2501.16466 Unpublished.

[23] Blake E Strom, Andy Applebaum, Doug P Miller, Kathryn C Nickels, Adam G Pennington, and Cody B Thomas. 2018. *MITRE ATT&CK: Design and Philosophy*. Technical Report. The MITRE Corporation, McLean, VA, USA. – pages.

[24] Mathew Vermeer, Michel van Eeten, and Carlos Gañán. 2022. Ruling the Rules: Quantifying the Evolution of Rulesets, Alerts and Incidents in Network Intrusion Detection. In *Proceedings of the 2022 ACM on Asia Conference on Computer and*

Communications Security (Nagasaki, Japan) *(ASIA CCS '22)*. Association for Computing Machinery, New York, NY, USA, 799–814. doi:10.1145/3488932.3517412

[25] Apurva Virkud, Muhammad Adil Inam, Andy Riddle, Jason Liu, Gang Wang, and Adam Bates. 2024. How does Endpoint Detection use the MITRE ATT&CK Framework?. In *33rd USENIX Security Symposium, USENIX Security 2024, Philadelphia, PA, USA, August 14-16, 2024*, Davide Balzarotti and Wenyuan Xu (Eds.). USENIX Association, Philadelphia, PA, USA, 19 pages. https://www.usenix.org/conference/usenixsecurity24/presentation/virkud

[26] Lingzhi Wang, Zhenyuan Li, Yi Jiang, Zhengkai Wang, Zonghan Guo, Jiahui Wang, Yangyang Wei, Xiangmin Shen, Wei Ruan, and Yan Chen. 2024. From Sands to Mansions: Towards Automated Cyberattack Emulation with Classical Planning and Large Language Models. arXiv:2407.16928 arXiv:2407.16928.

[27] Ming Xu, Hongtai Wang, Jiahao Liu, Yun Lin, Chenyang Xu, Yingshi Liu, Hoon Wei Lim, and Jin Song Dong. 2024. IntelEX: A LLM-driven Attack-level Threat Intelligence Extraction Framework. *CoRR* abs/2412.10872 (2024), 25 pages. arXiv:2412.10872 doi:10.48550/ARXIV.2412.10872

[28] Fan Yang, Jiacen Xu, Chunlin Xiong, Zhou Li, and Kehuan Zhang. 2023. PROGRAPHER: An Anomaly Detection System based on Provenance Graph Embedding. In *32nd USENIX Security Symposium, USENIX Security 2023, Anaheim, CA, USA, August 9-11, 2023*, Joseph A. Calandrino and Carmela Troncoso (Eds.). USENIX Association, Anaheim, CA, USA, 4355–4372. https://www.usenix.org/conference/usenixsecurity23/presentation/yang-fan

[29] Shakhzod Yuldoshkhujaev, Mijin Jeon, Doowon Kim, Nick Nikiforakis, and Hyungjoon Koo. 2025. A Decade-long Landscape of Advanced Persistent Threats: Longitudinal Analysis and Global Trends. In *Proceedings of the 2025 ACM SIGSAC Conference on Computer and Communications Security, CCS 2025, Taipei, Taiwan, October 13-17, 2025*, Chun-Ying Huang, Jyh-Cheng Chen, Shiuh-Pyng Shieh, David Lie, and Véronique Cortier (Eds.). ACM, Taipei, Taiwan, 3206–3220. doi:10.1145/3719027.3765085

[30] Bo Zhang, Yansong Gao, Changlong Yu, Boyu Kuang, Zhi Zhang, Hyoungshick Kim, and Anmin Fu. 2025. TAPAS: An Efficient Online APT Detection with Task-guided Process Provenance Graph Segmentation and Analysis. In *34th USENIX Security Symposium, USENIX Security 2025, Seattle, WA, USA, August 13-15, 2025*, Lujo Bauer and Giancarlo Pellegrino (Eds.). USENIX Association, Seattle, WA, USA, 607–624. https://www.usenix.org/conference/usenixsecurity25/presentation/zhang-bo-tapas

[31] Ágney Lopes Roth Ferraz, Sidnei Barbieri, Murray Evangelista de Souza, and Lourenço Alves Pereira Júnior. 2026. The Procedural Semantics Gap in Structured CTI: A Measurement-Driven STIX Analysis for APT Emulation. arXiv:2512.12078 [cs.CR] https://arxiv.org/abs/2512.12078

A Open Science

The camera-ready artifact for this paper is published at https://github.com/sidneibarbieri/sticks. The repository root exposes two reviewer-facing entry points. The first, `run_review_check.sh`, calls the canonical verifier at `measurement/sut/release_check.sh`. It reruns extraction, rebuilds figures, regenerates traceability tables, reevaluates the compatibility-validation packet, and compiles the manuscript. The second, `run_vm_backed_campaign.sh`, exposes the provider-aware lab path for a declared campaign/SUT pair from the same checkout.

The public artifact therefore supports both the canonical measurement-to-manuscript validation path and a heavier VM-backed realism path. The paper's reported measurement claims depend on the former. The latter is included to allow end-to-end reproduction of the published lab workflow under a controlled local substrate.

The artifact also exposes the materials most directly tied to the paper's claims: the campaign-level platform audit (`results/audit/campaign_platforms_software_only.csv`); the CVE outcome tables (`results/audit/campaign_cves.csv`); the compatibility assignments and rule breakdown (`results/audit/technique_compatibility.csv` and `results/audit/compatibility_rule_breakdown.csv`); the profile-specificity outputs (`results/group_specificity_analysis.json`); and the reviewer-facing traceability map (`measurement/sut/CLAIMS_TO_EVIDENCE.md`).

B Ethical Considerations

This paper studies how public structured CTI constrains adversary emulation environments. The contribution is measurement and evaluation guidance, not a new offensive capability. The released artifact operates on publicly available ATT&CK-style datasets and reproduces aggregate counts, audit tables, and deployability classifications; the reviewer-facing materials for this paper are measurement outputs and validation traces rather than a turnkey intrusion recipe.

The main dual-use concern is that concrete environment examples could be mistaken for deployment recipes. We mitigate that risk in two ways. First, the manuscript consistently treats SUT profiles as measurement artifacts rather than claims of historical replay fidelity. Second, the artifact releases controlled local lab automation for reviewer-owned infrastructure, conservative compatibility classes, and audit traces, but not a turnkey recipe for reproducing real intrusions against third-party systems. In case reviewers judge this work to raise additional ethical concerns, this appendix is intentionally explicit rather than silent.

C Generative AI Usage

Generative AI tools were used during manuscript preparation and artifact maintenance. They assisted with restructuring prose, proposing alternative phrasings, and refactoring support scripts and checks. The authors reviewed all AI-assisted text and code, reran the measurement pipeline after each substantive change, and manually verified that reported values, tables, figures, and citations remained grounded in the source datasets and released artifact. No reported result, count, or empirical claim is based solely on AI-generated content.

D Artifact-Linked Supplementary Material

The submission package includes additional audit tables, validation samples, and trace files generated by the same measurement pipeline described in Section 6. The main text already contains the nearest-neighbor CDF (Figure 8); this appendix adds the tactic-level compatibility view and anchors the corresponding artifact files for reviewers who want to inspect them directly.